

Merge Sorted Array

Company: Microsoft

Round: Online Assessment (OA)

Year: 2025

Difficulty: Easy

Topic: Arrays, Two Pointers

Source: LeetCode 88 – Merge Sorted Array

Problem Description

You are given two integer arrays `nums1` and `nums2`, both sorted in **non-decreasing order**.

You are also given two integers:

- $m \rightarrow$ number of valid elements currently present in `nums1`
- $n \rightarrow$ number of elements in `nums2`

The first m elements of `nums1` contain actual values.

The remaining n positions in `nums1` contain 0 and are provided as extra space to store the merged result.

We need to merge both arrays into a single sorted array and store the final result **inside `nums1` itself**.

The function should **not return** the merged array.

Input Format

- ☐ The first line contains two integers m and n — the number of valid elements in `nums1` and the number of elements in `nums2`, respectively.
- ☐ The second line contains $m + n$ space-separated integers denoting the array `nums1`. The first m elements are valid, and the remaining n elements are 0.
- ☐ The third line contains n space-separated integers denoting the array `nums2`.

Output Format

- ☐ Print $m + n$ space-separated integers on a single line — the merged array after combining `nums1` and `nums2` in non-decreasing order.

Constraints

- ☐ $0 \leq m, n \leq 200$
- ☐ $1 \leq m + n \leq 200$
- ☐ `nums1.length = m + n`
- ☐ `nums2.length = n`
- ☐ $-10^9 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^9$

Sample Input / Output

Example 1

Input:

```
nums1 = [1,2,3,0,0,0]
```

```
m = 3
```

```
nums2 = [2,5,6]
```

```
n = 3
```

Output:

```
[1,2,2,3,5,6]
```

Explanation

The actual elements in nums1 are:

```
[1,2,3]
```

and nums2 contains:

```
[2,5,6]
```

After merging:

```
[1,2,2,3,5,6]
```

The result is stored directly in nums1.

Example 2

Input:

```
nums1 = [1]
```

```
m = 1
```

```
nums2 = []
```

```
n = 0
```

Output:

```
[1]
```

Explanation

nums2 is empty, so there is nothing to merge.

Example 3

Input:

```
nums1 = [0]
```

```
m = 0
```

```
nums2 = [1]
```

```
n = 1
```

Output:

```
[1]
```

Explanation

nums1 has no valid elements because $m = 0$.

Therefore, we simply place the element from nums2 into nums1.

Approach to Solve This Problem:

The best approach is to use the **Two Pointer technique** and merge the arrays **from the end**.

We use three pointers:

```
i = m - 1
```

```
j = n - 1
```

```
k = m + n - 1
```

Where:

- $i \rightarrow$ points to the last valid element in nums1
- $j \rightarrow$ points to the last element in nums2
- $k \rightarrow$ points to the last available position in nums1

For:

```
nums1 = [1,2,3,0,0,0]
```

```
nums2 = [2,5,6]
```

Initially:

```
i = 2
```

```
j = 2
```

```
k = 5
```

We compare:

nums1[i] and nums2[j]

and place the **larger element** at nums1[k].

Why Do We Start From the End?

This is the key idea.

nums1 already contains elements that we need to preserve.

If we start merging from the beginning, we may overwrite elements that have not yet been processed.

Instead, we start from the end because nums1 has empty space at the end.

For example:

nums1 = [1,2,3,0,0,0]

↑

empty space

The largest element can safely be placed at the last position.

So:

3 vs 6

6 is larger:

[1,2,3,0,0,6]

Then:

3 vs 5

5 is larger:

[1,2,3,0,5,6]

Then:

3 vs 2

3 is larger:

[1,2,3,3,5,6]

Continue until all elements are merged.

Final result:

[1,2,2,3,5,6]

Java Solution:

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {

        int i = m - 1;    // Last valid element of nums1
        int j = n - 1;    // Last element of nums2
        int k = m + n - 1; // Last position of nums1

        while (j >= 0) {

            if (i >= 0 && nums1[i] > nums2[j]) {
                nums1[k] = nums1[i];
                i--;
            } else {
                nums1[k] = nums2[j];
                j--;
            }

            k--;
        }
    }
}
```

Solution:

- ❑ **Read the input:** main() reads m and n, then reads m + n integers into nums1 and n integers into nums2, matching the Input Format.
- ❑ **Initialize three pointers:** Set i = m - 1 to point to the last valid element of nums1, j = n - 1 to point to the last element of nums2, and k = m + n - 1 to point to the last position of nums1.
- ❑ **Compare elements from the end:** While j >= 0, compare nums1[i] and nums2[j]. If i >= 0 and nums1[i] > nums2[j], place nums1[i] at nums1[k] and move i left. Otherwise, place nums2[j] at nums1[k] and move j left.
- ❑ **Move the output pointer:** After placing an element, decrement k to move to the previous available position in nums1. This ensures that elements are merged from right to left without overwriting the unprocessed elements of nums1.
- ❑ **Finish the merge:** Continue the process until all elements of nums2 have been placed. If elements remain in nums1, they are already in their correct positions, so no additional copying is required.
- ❑ **Print the result:** Print all m + n elements of nums1 space-separated on a single line, matching the Output Format.

Complexity Analysis:

Time Complexity

$O(m + n)$

Every element from both arrays is processed at most once.

Space Complexity

$O(1)$

No extra array is created. The result is stored directly inside nums1.
